

Even though it sounds like an over-simplification, a smart contract can be seen as a contract drafted by a programmer using programs and protocols, rather than by a lawyer in the traditional way. Please note, as mentioned earlier, blockchain technology nowadays is not just about cryptocurrencies like Bitcoin. The widespread acceptance of Solidity and the Ethereum platform by the general programming community might take some time. Users of the Ethereum platform may find it useful to learn Solidity, which operates in a way that's similar to Java — the source code is compiled to byte code and then a virtual machine is used to run it.

First, let us try to install the compiler for Solidity. I was able to install one of its compilers called *solc* using the package manager Snap.

The following command installed the compiler *solc* in my system:

```
sudo snap install solc
```

Solidity programs use the extension *.sol*. I searched the Internet in vain to find a *HelloWorld.sol* program. I even tried the website [www.helloworldcollection.de](http://www.helloworldcollection.de), which offers 'Hello World' programs in hundreds of different programming languages. But there was no Solidity program and there is an opportunity there. You can be the first one to add a 'Hello World' program in the Solidity programming language.

Again, this tells us something about blockchain programming languages. We must not treat them like other traditional programming languages. Even the smallest Solidity program available was relatively large, so I ended my use of the Solidity compiler *solc* with the command:

```
solc --license
```

...on the terminal.

Figure 2 shows the output of the command, which tells us that most of the source code of Solidity is

```
# solc --license
Most of the code is licensed under GPLv3 (see below), the license for individual
parts are as follows:
```

Figure 2: The Solidity compiler in action

licensed under the GNU General Public License. But please note that, based on the philosophy of blockchain, it is implicit that any programming language that can be used in the blockchain environment should have an open source implementation. A technology that depends on openness and decentralised behaviour can support nothing but an open source paradigm — it doesn't make any sense otherwise. Without further ado, let us move on to some more blockchain programming languages that might become trends in the near future.

**Simplicity:** This blockchain programming language is supported by Blockstream, a blockchain technology company. Simplicity is dependent on two other traditional software — Haskell (a functional programming language) and *coq* (an interactive theorem prover). Simplicity is also used to generate smart contracts.

**Vyper:** This is another programming language that tries to target the Ethereum platform, similar to Solidity. Vyper calls itself a Pythonic programming language — which in simple terms means it exploits the features of Python.

**CX:** This is a programming language supported by Skycoin, a blockchain technology company. CX claims to be a general-purpose programming language.

**Scilla:** This language is mostly used to draft smart contracts. It also takes its strength from the proof assistant software *coq*.

**Rholang:** This blockchain programming language is still in its development stages (according to the

Rholang source on GitHub).

**Obsidian:** This is another blockchain programming language used for drafting smart contracts.

Of course, I could have added many more to this list. However, as I am not an expert in any of these programming languages, I think it is better to stop at this point. But as a programming language enthusiast, I have to make two observations before I conclude this article. First, I think almost all the so called blockchain programming languages are still evolving and may take a few more years to reach their true potential. Second, if you observe carefully, almost all these languages target a specific blockchain technology platform. This makes the widespread acceptance of any one particular programming language rather difficult. At the same time, with sufficient number of fans and followers, any one of these programming languages could make a huge impact in the industry.

My sole intention in writing this article is to throw some light on the fact that there are dedicated programming languages to work with blockchains. I leave readers with the important question: "As a blockchain enthusiast, should I learn one of these new programming languages or should I just stick with the traditional ones like C++, Python, JavaScript, etc?" Frankly, I don't know the answer to this. With blockchain technology becoming more and more popular, this is an important question that needs an answer. **END** 

 By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. He maintains a technical blog at [www.computingforbeginners.blogspot.in](http://www.computingforbeginners.blogspot.in).